



# **Formação Backend: Python & FastAPI — O Caminho Sannin de Orochimaru**

**Projeto Mushi Bingo**

# Aula Anterior

- Configuração do SQLAlchemy (engine, Session, Base)
- Criação dos models (Autor, Estudio, Anime) com relacionamentos
- Alembic: primeira migration
- Organização de rotas com APIRouter
- Persistência dos endpoints de `/animés` no banco de dados

## Injeção de Dependência (Depends)

Na aula anterior, cada endpoint criava sua própria `Session` manualmente. A Injeção de Dependência resolve essa repetição: o FastAPI cuida de criar (e fechar) a sessão para cada requisição.

- Evita duplicação de código
- Centraliza a lógica de acesso ao banco
- Facilita testes (a dependência pode ser substituída)

## Criando a dependência da Sessão

```
# app/database.py

def get_session():
    with Session() as session:
        yield session
```

```
# app/routers/animes.py

from typing import Annotated

from fastapi import Depends
from sqlalchemy.orm import Session as SQLAlchemySession

from app.database import get_session

DbSession = Annotated[SQLAlchemySession, Depends(get_session)]
```

## Refatorando os endpoints com Depends

```
# app/routers/animés.py

@router.post("/", status_code=HTTPStatus.CREATED, response_model=AnimePublic)
def create_anime(anime: AnimeSchema, session: DbSession):
    db_anime = Anime(**anime.model_dump())
    session.add(db_anime)
    session.commit()
    session.refresh(db_anime)

    return db_anime

@router.get("/", response_model=AnimeList)
def read_animés(session: DbSession):
    animés = session.query(Anime).all()

    return {"animés": animés}
```

**Dica:** repita esse padrão em `/autores` e `/estudios`.

# RedirectResponse

Redireciona o cliente para outra URL. Útil, por exemplo, após criar um recurso enviar o cliente direto para o novo registro.

```
# app/routers/animas.py

from fastapi.responses import RedirectResponse

@router.post("/", status_code=HTTPStatus.CREATED)
def create_anime(anime: AnimeSchema, session: DbSession):
    db_anime = Anime(**anime.model_dump())
    session.add(db_anime)
    session.commit()
    session.refresh(db_anime)

    return RedirectResponse(
        url=f"/animas/{db_anime.id}", status_code=HTTPStatus.SEE_OTHER
    )
```

# Middleware

Um middleware intercepta **toda** requisição antes (e/ou depois) dela chegar ao endpoint. Útil para logs, métricas, headers, autenticação, entre outros.

```
# app/main.py

import time

@app.middleware("http")
async def log_requests(request: Request, call_next):
    inicio = time.time()
    response = await call_next(request)
    duracao = time.time() - inicio

    print(f"{request.method} {request.url.path} – {duracao:.3f}s")

    return response
```

## Tratamento de exceções mais robusto

Em vez de tratar erro por erro dentro de cada endpoint, centralizamos com `exception_handler`.

```
# app/main.py

from http import HTTPStatus

from fastapi.responses import JSONResponse
from sqlalchemy.exc import IntegrityError

@app.exception_handler(IntegrityError)
def integrity_error_handler(request, exc):
    return JSONResponse(
        status_code=HTTPStatus.BAD_REQUEST,
        content={"detail": "Violação de integridade (FK inválida ou campo único duplicado)."},
    )
```





## Atividade

- Aplique `Depends(get_session)` também em `/autores` e `/estudios`
- Adicione o middleware de log ao projeto e observe o tempo de cada requisição no terminal
- Teste enviar um `autor_id` inexistente ao criar um anime e veja o `exception_handler` em ação

# Próximos tópicos

- `UploadFile` e `File`
- Upload e validação da capa do anime
- Servindo arquivos estáticos
- Revisão geral do projeto

# Referencias

- [FastAPI — Dependencies](#)
- [FastAPI — Middleware](#)
- [FastAPI — Handling Errors](#)
- [SQLAlchemy — Exceptions](#)
- [FastAPI do Zero](#)



Até a próxima!